57

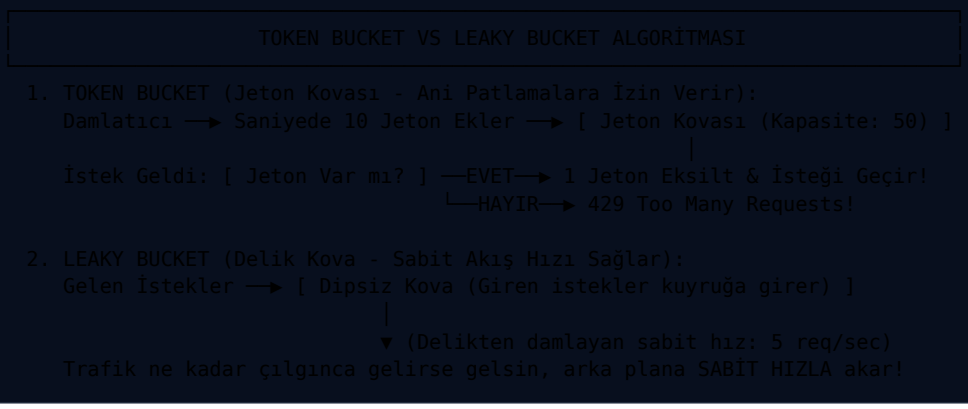
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 3: Dağıtık Token-Bucket & Leaky-Bucket Hız Sınırlayıcı (Rate Limiter)

MAANG Case 3: Distributed Token-Bucket & Leaky-Bucket Rate Limiter

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Token Bucket bir lunapark biletçisi gibidir. Kovaya her saniye 1 bilet atılır (en fazla 10 bilet birikir). Bir anda 5 arkadaş gelirseniz kovadaki 5 bileti alıp hemen dönme dolaba binebilirsiniz (patlamalı trafiğe izin verir). Bilet biterse kapıda kalırsınız. Leaky Bucket ise altı delik bir su kovasıdır. Kovaya yukarıdan bir kova suyu birden döksen de altındaki minik delikten su damla damla, sabit ve sakin bir hızla akar; arka plandaki narin veritabanını asla boğmaz!

🇬🇧 ENGLISH (ELI5)

Token Bucket is an arcade coin dispenser. It drops 1 token every second into a cup that holds up to 10 tokens. A burst of 5 gamers can grab 5 tokens simultaneously and play immediately (burst-tolerant). Once empty, newcomers are rejected. Leaky Bucket is a funnel with a tiny hole at the base. Even if a sudden flood of 100 gallons dumps into the funnel, water drips out of the bottom at a strictly constant rate of 1 drop/second, shielding fragile downstream services from traffic spikes.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Mülakatında Algoritma Karşılaştırması ve Kilit-Siz Uygulama: 1) Token Bucket Mantığı: İki ana değişken vardır: `capacity` ve `refillRate`. Her saniye bir zamanlayıcı thread çalıştırmak yerine 'Tembel Doldurma' (Lazy Refill) formülü uygulanır: `yeniJeton = min(kapasite, mevcutJeton + (simdikiZaman - sonYenilemeZamani) × refillRate)`. Bu sayede kova kontrolü yapay zeka gibi sıfır arka plan maliyetiyle yürütülür. 2) CAS ile Eşzamanlılık: `AtomicLong` veya `AtomicReference` kullanılarak jeton harcama işlemi kilit almadan tek çevrimde gerçekleştirilir. 3) Leaky Bucket Mantığı: Genellikle `ArrayBlockingQueue` veya Redis kuyruğu ile modellenir; gelen istekler kuyruğa alınır, bir tüketici thread sabit hızda (`rate`) kuyruktan iş çeker.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Algorithmic Breakdown: 1) Token Bucket Optimization: Avoid dedicated timer background threads. Implement 'Lazy Refill': compute accumulated tokens mathematically on-demand: `newTokens = min(capacity, tokens + (now - lastRefill) × rate)`. 2) Lock-Free State Mutation: Encapsulate token counts and timestamps in an immutable Java Record updated atomically via `AtomicReference.compareAndSet()`. 3) Leaky Bucket Topology: Models traffic shaping. Incoming requests enter a bounded FIFO buffer; an internal rate-governor drains requests at an uncompromising cadence, smoothing bursty traffic profiles before downstream ingress.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Lock-Free ve Tembel Doldurmalı Token Bucket Rate Limiter
import java.time.Instant;
import java.util.concurrent.atomic.AtomicReference;

public class AtomicTokenBucketLimiter {
    private final long capacity;
    private final double refillTokensPerSecond;

    // Durumu değişmez bir record içinde atomik tut
    private record State(long tokens, long lastRefillNanos) {}
    private final AtomicReference<State> state;

    public AtomicTokenBucketLimiter(long capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.state = new AtomicReference<>(new State(capacity, System.nanoTime()));
    }

    public boolean tryAcquire(long tokensRequested) {
        while (true) {
            State current = state.get();
            long now = System.nanoTime();
            long elapsedNanos = Math.max(0, now - current.lastRefillNanos());
            double newTokens = (elapsedNanos / 1_000_000_000.0) * refillTokensPerSecond;

            long updatedTokens = Math.min(capacity, current.tokens() + (long) newTokens);

            if (updatedTokens < tokensRequested) {
                return false; // Jeton yetersiz: İstek reddedildi (429)!
            }

            State next = new State(updatedTokens - tokensRequested, now);
            if (state.compareAndSet(current, next)) {
                return true; // Atomik olarak jeton düşüldü: İstek geçti!
            }
        }
    }

    void main() {
        var limiter = new AtomicTokenBucketLimiter(5, 2.0); // 5 kapasite, sn'de 2 jeton
        for (int i = 1; i <= 7; i++) {
            boolean allowed = limiter.tryAcquire(1);
            System.out.printf("İstek #%d Kabul Edildi mi? %s%n", i, allowed ? "✅ EVET" : "❌ 429
RED");
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Dağıtık sistemlerde (ör. Kubernetes üzerinde 50 pod) lokal RAM'deki Token Bucket işe yaramaz. Durumu Redis üzerinde Lua Script ile atomik yürütmeli veya NGINX / Envoy API Gateway katmanında merkezileştirmelisiniz.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Local in-memory rate limiters fail across horizontal deployments (e.g. 50 microservice pods). Synchronize global rate limiting states using atomic Redis Lua scripts or offload enforcement to edge gateways like Envoy or Cloudflare.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Token Bucket ani trafik patlamalarına izin verir (lazy refill ile kilit-siz hesaplanır). Leaky Bucket ise trafiği sabit hızda damlatarak pürüzsüzleştirir.

🎯 Key Takeaways for the Interview

• Token Bucket tolerates bursty ingress computed via lock-free lazy refills. Leaky Bucket smooths spikes into a constant-rate outflow.